Université des Sciences et Technologies de Houari Boumediene

Faculté d'informatique

2e année d'ingénieur – Section B

UniTrans

Un système de gestion des transports universitaires.

Projet Finale de Base de Données 2025/2026

Analyse | Modélisation | Implémentation | Normalisation

Compilé et développé PAR :

Martin CHAUKE
23238ZWE13686
Groupe 1

Brighton MATIKITI
23238ZWE13691
Groupe 4

Supervisé par :

Dr LAHRECHE Abdelmadjid

Introduction

Avec l'avancée rapide des technologies de l'information, les applications web sont devenues des outils essentiels pour gérer efficacement des systèmes complexes. Les universités, en particulier, font face à des défis croissants pour organiser les services de transport des étudiants en raison de la demande croissante, de la complexité des horaires et de la nécessité de coordination en temps réel.

Les méthodes traditionnelles de gestion des transports reposent souvent sur des processus manuels, ce qui peut entraîner des inefficacités, un manque de transparence et des difficultés dans le suivi des opérations. Pour relever ces défis, il est nécessaire d'un système structuré et automatisé qui garantisse fiabilité, évolutivité et facilité d'utilisation.

Ce projet introduit *UniTrans*, un système universitaire de gestion des transports conçu pour simplifier la gestion des bus, des chauffeurs, des lignes de transport, des incidents et des abonnements étudiants. Le système offre une plateforme centralisée qui permet aux administrateurs de gérer efficacement les ressources de transport tout en offrant aux étudiants un moyen simple de s'abonner et d'accéder aux services de transport.

De plus, la conception du système suit la méthodologie de modélisation conceptuelle et logique des données, garantissant une base de données bien structurée et normalisée. Cette approche garantit l'intégrité des données, réduit la redondance et facilite l'expansion future du système.

L'objectif de ce projet est de fournir une solution efficace, évolutive et conviviale pour la gestion des systèmes de transport universitaires.

Analyse/Aperçu du système

Utilisateurs du Système.

Étudiant

Peuvent créer un compte, s'abonner aux services de transport, consulter les lignes et plannings assignés, et gérer leur abonnement.

Pilote

Peut consulter les trajets assignés, surveiller les horaires et signaler les incidents survenant pendant le service.

Responsable / Administrateur des transports

Il a un contrôle total sur le système, notamment : la gestion des élèves, la gestion des chauffeurs, la gestion des bus, la gestion des lignes et des gares, la création d'horaires, l'attribution des bus et des chauffeurs, le suivi des abonnements, la revue des incidents et la gestion des attributions des lignes.

UniTrans – University Bus Transport Management System

System Architecture

FRONTEND LAYER

1. MANAGER PORTAL (Next.js) – Port 3000
Admin / Transport Staff Interface
Tech: React, TypeScript, Axios, Context API

- Dashboard
- Students Management
- Buses Management
- Drivers Management
- Lines Management
- Trips Management
- Incidents Management
- Subscriptions Management
- Statistics & Analytics
- Reports Generation

2. STUDENT PORTAL (Next.js) – Port 3001
Student Self-Service Interface
Tech: React, TypeScript, Axios, Context API

- My Assigned Transport
- My Schedules
- My Subscriptions
- Subscription History
- Request Line Change
- My Notifications
- Profile Management
- Announcements & Alerts
- Help & Support

3. DRIVER PORTAL (Next.js) – Port 3002
Driver / Field Operations Interface
Tech: React, TypeScript, Axios, Context API

- Today's Trips
- Assigned Bus
- Route / Line Details
- Trip Status Start / Complete / Delay
- Passenger / Seat Context
- Incidents Reporting
- Notifications (Dispatch / Changes)
- Profile Management
- Help & Support

HTTPS / REST API (JSON)

LocalStorage (Manager Namespace)
access_token_manager
refresh_token_manager

LocalStorage (Student Namespace)
access_token_student
refresh_token_student

LocalStorage (Driver Namespace)
access_token_driver
refresh_token_driver

BACKEND LAYER

DJANGO REST FRAMEWORK API – Port 8000

JWT Authentication
- Access / Refresh Tokens
- Refresh Rotation
- Token Blacklist (Logout)

CORS
Enabled for Origins: :3000, :3001, :3002

Permissions & Role-Based Access
(Manager / Student / Driver)

API Documentation
- Swagger UI
- ReDoc

Global Exception Handling & Logging
- Request / Response Logs
- Error Tracking

DJANGO APPLICATIONS (9)

1. Accounts
- Registration
- Authentication
- User (Role)
- Student Profile
- Driver Profile
- JWT Management

2. Lines
- Routes (Lines)
- Stations
- Line–Station Mapping

3. Schedules
- Timetables
- Schedule Definitions

4. Buses
- Fleet Management
- Bus–Line Assignments
- Driver Linkage (where applicable)

5. Trips
- Trip Planning
- Trip Execution
- Rows
- Seat Assignments
- Trip Status
- Driver (Trip)

6. Subscriptions
- Student Subscriptions
- Subscription History
- Renewal Tracking
- Audit Trail

7. Incidents
- Report Incidents
- Track Issues
- Status Updates
- Reported By: Student / Driver

8. Notifications
- Alerts & Messages
- Triggers From:
  - Subscriptions
  - Incidents
  - Trip Updates
- Delivery Rules to Students & Drivers

9. Reports
- Analytics
- Aggregation From:
  - Trips
  - Subscriptions
  - Incidents
  - Driver / Trip KPIs

Core DRF Components
Serializers (JSON) • ViewSets / API Endpoints • Filters / Pagination • Throttling • Permissions

INTEGRATIONS / SERVICES
- Email Service (SMTP)
- SMS Gateway
- Push Notifications
- File Storage (MinIO / S3 Compatible)
- Logging & Monitoring (Prometheus / Grafana)

DATABASE LAYER

PostgreSQL 14+ Database (Single Database for All Apps)

users
PK user_id
name
email (unique)
password
role (manager/ student/driver)
is_active
is_staff
date_joined

students
PK student_id
FK user_id
registration_number (unique)
first_name
last_name
email
phone

drivers
PK driver_id
FK user_id
license_number (unique)
first_name
last_name
phone
assigned_bus_id (FK)

lines
PK line_id
name
description
is_active

stations
PK station_id
name
location
latitude
longitude

line_stations
PK line_station_id
FK line_id
FK station_id
order_index
UNIQUE (line_id, station_id)

buses
PK bus_id
registration_number (unique)
model
capacity
status

bus_assignments
PK bus_assignment_id
FK bus_id
FK line_id
start_date
end_date
notes

schedules
PK schedule_id
FK line_id
day_of_week
departure_time
arrival_time
direction

trips
PK trip_id
FK schedule_id
FK bus_id
FK driver_id (nullable)
actual_departure
actual_arrival
status

rows
PK row_id
FK bus_id
row_number
seat_count
UNIQUE (bus_id, row_number)

seat_assignments
PK seat_assignment_id
FK trip_id
FK row_id
FK student_id
seat_number
assigned_at
UNIQUE (trip_id, row_id, seat_number)

subscriptions
PK subscription_id
FK student_id
FK line_id
start_date
end_date
is_active

subscription_histories
PK subscription_history_id
FK student_id
FK old_line_id
FK new_line_id
change_date
reason

incidents
PK incident_id
FK trip_id (nullable)
reported_by_type (student/driver)
reported_by_id (nullable)
title
description
status
reported_at
resolved_at

notifications
PK notification_id
FK user_id
notification_type
title
message
is_read
created_at
related_entity_type
related_entity_id

student_reports
PK report_id
FK student_id
report_type
title
description
status
submitted_at
resolved_at

report_aggregates
PK aggregate_id
metric_type
period_start
period_end
value
generated_at
generated_by

DATA FLOW (REQUEST / RESPONSE)

1. User logs in via any portal (Manager / Student / Driver).
2. Credentials sent to Auth API.
3. API validates and returns JWT (access + refresh).
4. Frontend stores tokens in portal-specific localStorage.
5. Subsequent requests include:
   Authorization: Bearer <access>
6. API validates JWT, checks role (manager / student / driver) and permissions.
7. Business logic executed via Django ORM.
8. JSON response serialized and returned; UI renders data.

AUTHENTICATION FLOW

1. Login request → returns access + refresh tokens.
2. Access token used for API requests.
3. When expired → frontend sends refresh token to refresh endpoint.
4. New access (and rotated refresh) token returned.
5. On logout → refresh token is blacklisted.
6. Each portal uses its own localStorage namespace:
   - Manager: access_token_manager / refresh_token_manager
   - Student: access_token_student / refresh_token_student
   - Driver: access_token_driver / refresh_token_driver
   → Allows concurrent logins without collision.

KEY RELATIONSHIPS

- User → Student & Driver (profile children).
- Line → Station, Timetable (Schedules).
- Bus → BusAssignment → Line.
- Bus → Trip.
- Trip → Row → SeatAssignment → Student.
- Trip ↔ Driver (operational assignment).
- Student → Subscription → Line.
- Student → Incident / Notification.
- Driver → Trip / Bus (as assigned).
- Driver → Notification.
- Reports aggregate Trip, Subscription, Incident (and Driver / Trip KPIs).

LEGEND

- Component / Module
- Use Case / Feature
- Database / Storage
- Synchronous (HTTPS / REST)
- Async / Event Trigger (Notifications, Jobs)
- 1 — One
- 1..* — One to Many
- 0..* — Zero to Many
- Aggregation / Derived

Figure 1: Architecture système globale d'Unitrans

Règles métier (Régles des contraintes et Intégrités).

1. Un étudiant s'abonne à une ligne de transport et non directement à un bus.
2. Un étudiant peut avoir plusieurs abonnements au fil du temps
3. Chaque abonnement relie un étudiant à une ligne de transport.
4. SubscriptionHistory enregistre les modifications, renouvellements et annulations d'abonnement.
5. Une ligne de transport dessert plusieurs stations dans un ordre défini.
6. Une ligne de transport peut avoir plusieurs bus attribués au fil du temps.
7. Un bus peut être affecté à différentes lignes de transport au fil du temps.
8. Un bus contient plusieurs rangées.
9. Chaque rangée contient plusieurs sièges.
10. Les chauffeurs peuvent être affectés à différents bus au fil des heures. (Mais un conducteur ne peut conduire qu'un bus pendant la période assignée et peut être affecté à un autre).
11. Un bus peut avoir des chauffeurs différents au fil du temps.
12. Chaque voyage est généré à partir d'un planning.
13. Un bus peut effectuer plusieurs trajets.
14. Chaque trajet est effectué par un conducteur et un bus.
15. Les incidents sont associés à un voyage spécifique.
16. Les élèves peuvent se voir attribuer des places pour des voyages spécifiques.
17. Les utilisateurs peuvent exercer l'un des rôles suivants : administrateur, responsable des transports, élève et chauffeur.

Modèle de données conceptuelles (MCD) : Association modèle-entité.

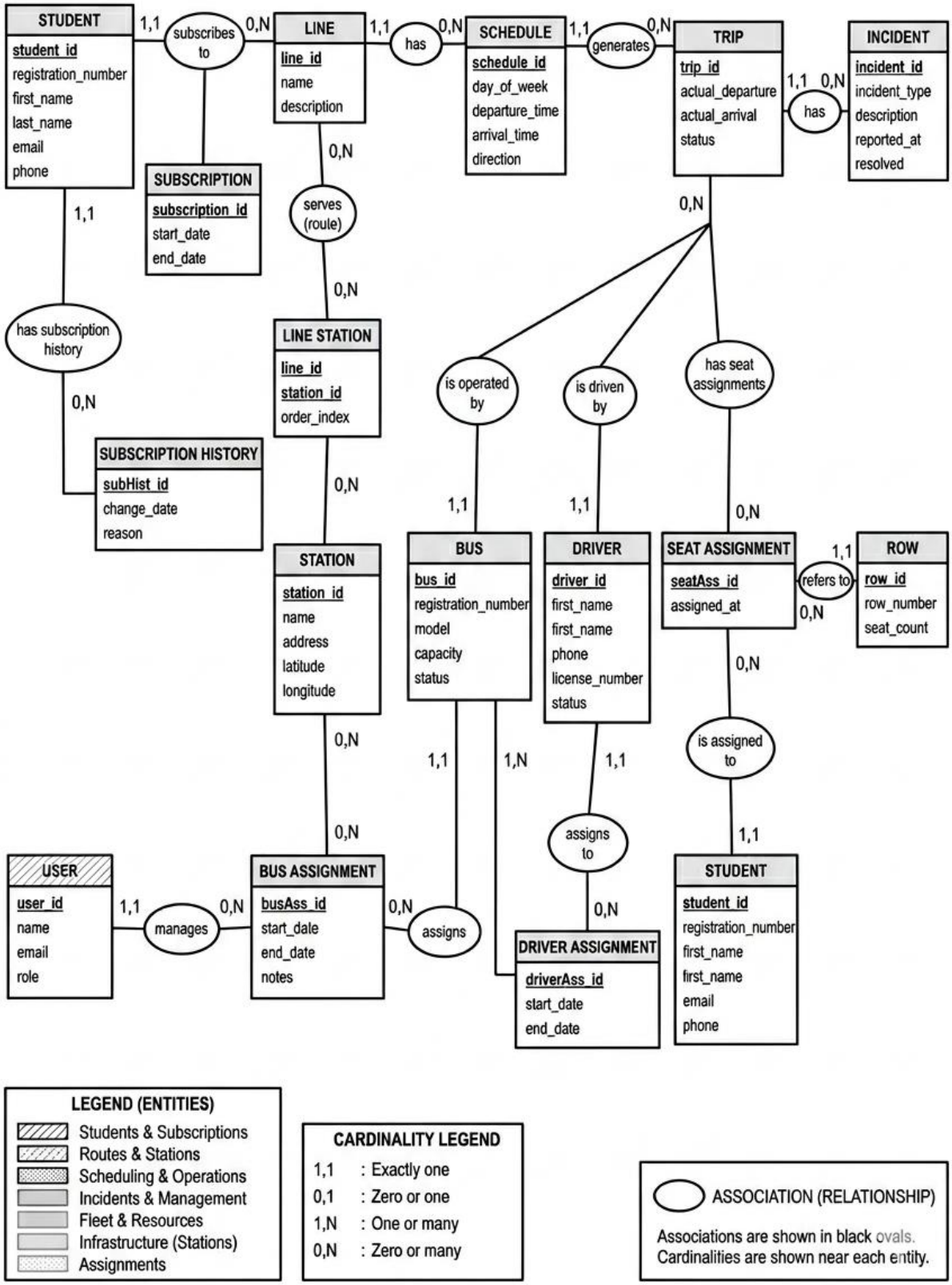
Les Entités.

1. *Étudiant* : Représente un étudiant universitaire utilisant les services de transport.
2. *Chauffeur :* représente un chauffeur chargé d'opérer des bus.
3. *Utilisateur :* Représente les utilisateurs authentifiés du système.

   Rôles :

   Administrateur, responsable des transports, élève et chauffeur.
4. *Bus :* Représente un véhicule pouvant être assigné aux lignes de transport au fil du temps.
5. *Rangée* : Représente les rangées de sièges à l'intérieur d'un bus.
6. *Ligne* : Représente une route de transport.
7. *Station :* Représente les arrêts desservis par les lignes de transport.
8. *Horaire* : Représente les horaires de départ et d'arrivée prévus pour une ligne.
9. *Trip :* Représente une exécution réelle du transport générée à partir d'un planning.
10. *Abonnement* : Représente l'inscription des étudiants sur une ligne de transport.
11. *AbonnementHistorique* : Représente les modifications, renouvellements et annulations d'abonnements.
12. *Incident* : Représente un problème signalé lors d'un trajet.
13. *BusAssignment* : Représente l'attribution temporelle des bus aux lignes de transport.
14. *Assignation du conducteur :* Représente l'affectation temporelle des conducteurs aux bus.
15. *Attribution de sièges* : Représente l'attribution des places des élèves lors des déplacements.
16. *DESSERVS* : Entité associative reliant les lignes et stations à l'ordre des itinéraires.

Figure 2: Modèle conceptuel de données d'Unitrans.



Relational Model/Modéle-Relationele De Données(MLD).

Utilisateur (*user_id*, nom, email, rôle)

Étudiant(*student_id*, registration_number, first_name, last_name, email, téléphone, *#user_id*)

Conducteur(*driver_id*, first_name, last_name, téléphone, license_number, statut, *#user_id*)

Ligne (*line_id*, nom, description)

Station (*station_id*, nom, adresse, latitude, longitude)

Bus (*bus_id,* registration_number, modèle, capacité, statut)

Rang(*row_id*, row_number, seat_count, *#bus_id*)

Planning (*schedule_id*, day_of_week, departure_time, arrival_time, direction, *#line_id*)

Trip(*trip_id,* actual_departure, actual_arrival, statut, *#schedule_id*, *#bus_id*, *#driver_id*)

Incident (incident_id, incident_type, description, reported_at, résolu, *#trip_id*)

Abonnement(subscription_id, start_date, end_date, *#student_id*, *#line_id*)

SubscriptionHistory(*subscription_history_id*, change_date, action_type, reason, *#student_id*, *#line_id*)

BusAssignment(bus_assignment_id, start_date, end_date, notes, *#bus_id*, *#line_id*, *#user_id*)

AssignationConducteur (*driver_assignment_id,* start_date, end_date, *#driver_id*, *#bus_id*)

AssignationSiège(*seat_assignment_id*, assigned_at, *#student_id*, *#trip_id*, *#row_id*)

SERVS*(#line_id*, *#station_id*, order_index)

Note : L'attribut student_id est utilisé comme clé primaire au lieu de la registration_number. Cette conception garantit un identifiant unique, stable et généré par le système, indépendant des changements externes ou administratifs, tandis que le registration_number reste un attribut unique mais non primaire.

PORTÉE DE L'IMPLÉMENTATION

Clés primaires.

Chaque entité contient un identifiant unique :

- user_id
- student_id
- driver_id
- line_id
- station_id
- bus_id
- row_id
- schedule_id
- trip_id
- incident_id
- subscription_id
- subscription_history_id
- bus_assignment_id
- driver_assignment_id
- seat_assignment_id

Clés étrangères

Indiqué bleu dans le modèle relationnel. (Voir plus haut.)

Normalisation

La base de données Unitrans satisfait :

Première forme normale (1NF) : Tous les attributs sont atomiques.

Deuxième forme normale (2NF) : Tous les attributs non clés dépendent entièrement de la clé primaire et il n'existe pas de dépendances fonctionnelles partielles.

Troisième forme normale (3NF) : Aucune dépendance transitive n'existe. Par exemple : les informations sur les conducteurs dépendent uniquement de *driver_id*, le statut d'abonnement dépend uniquement de *subscription_id,* et les données d'attribution de bus dépendent uniquement de *bus_assignment_id*.

BCNF : Toute dépendance fonctionnelle a une clé candidate comme déterminant.

DESCRIPTION DE L'APPLICATION ET CONCEPTION

Portail Manager/Admin

S'occupe de la gestion des utilisateurs, de la gestion des élèves, de la gestion des conducteurs, de l'affectation des bus, de l'affectation des conducteurs, de la gestion des horaires, de l'examen des incidents et des statistiques.

Portail étudiant

Gère l'inscription/connexion, l'abonnement, le renouvellement de l'abonnement, l'attribution des places et la consultation des horaires.

Portail pilote

S'occupe des trajets et bus assignés, des détails de l'itinéraire/ligne et du signalement d'incident.

Diagramme de cas d'utilisation

Résume le flux et la fonctionnalité de l'application.

Figure 3: Diagramme de cas d'utilisation

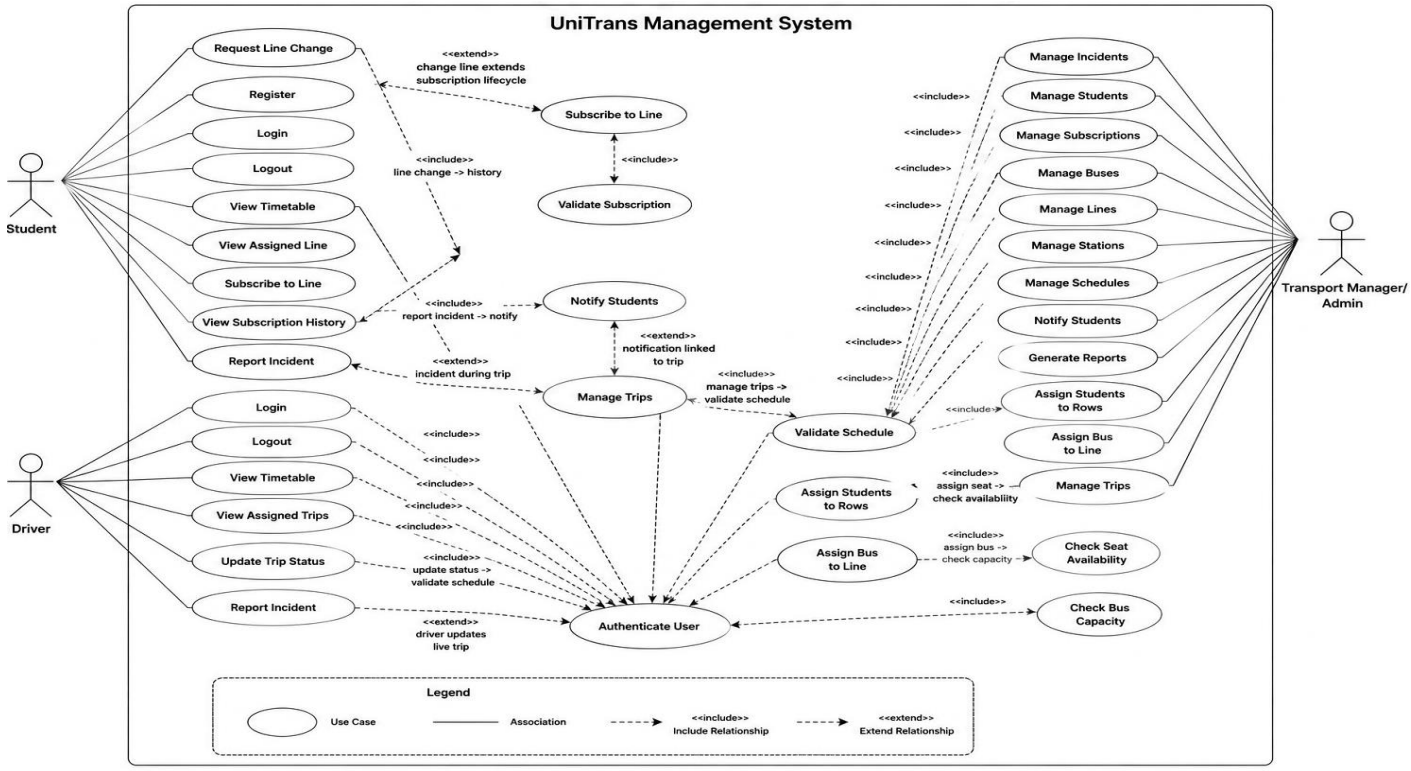
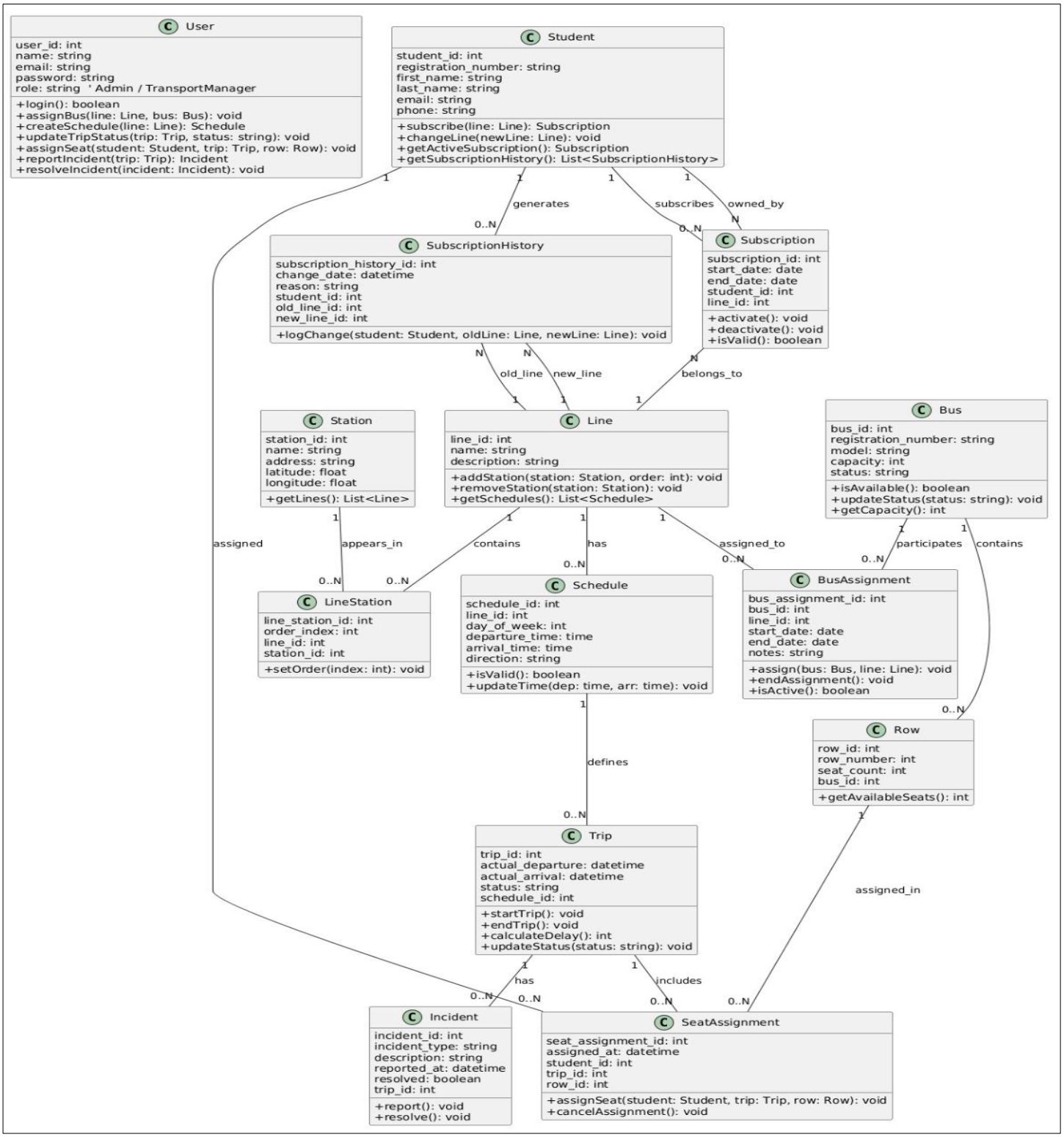


Figure 4: Schéma de classes de l'application Unitrans



INTERFACES D'APPLICATION

Interfaces d'authentification.

Étudiant

Figure 5: Page d'accueil des étudiants

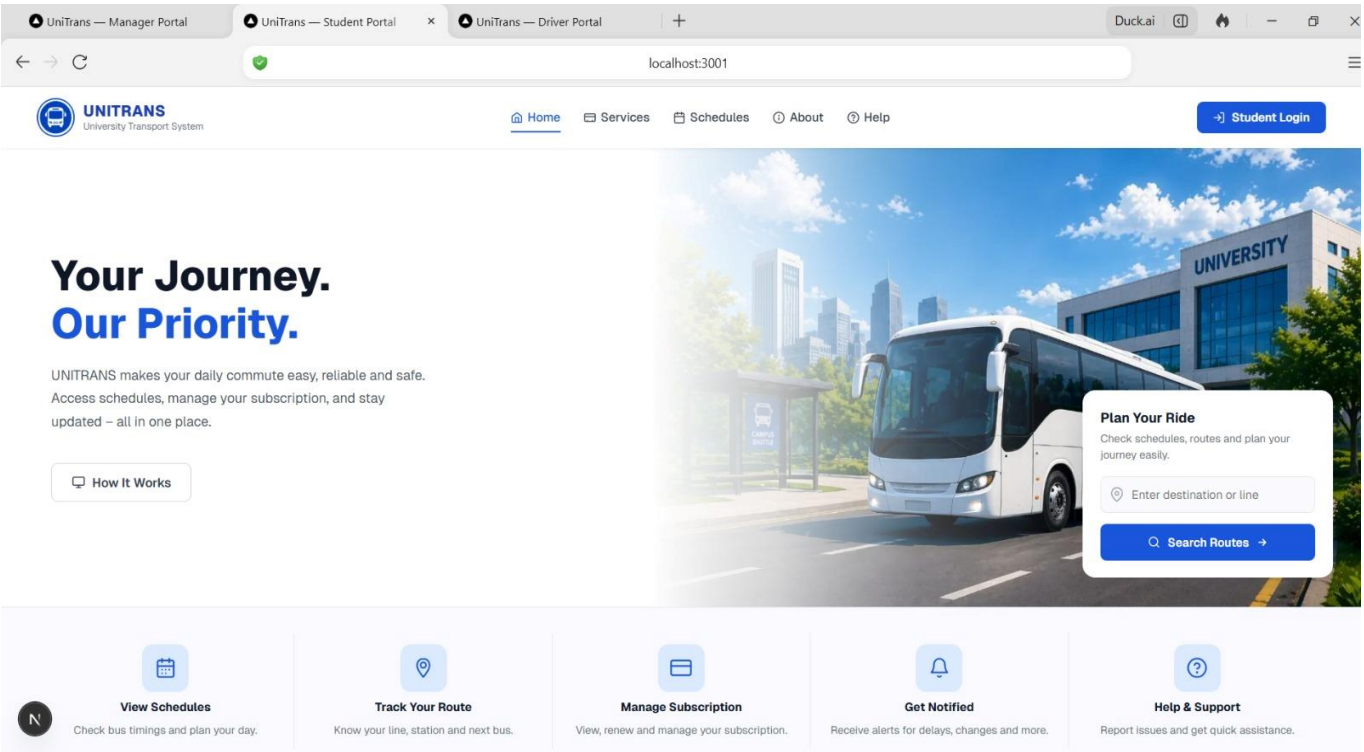


Figure 6: Connexion étudiante

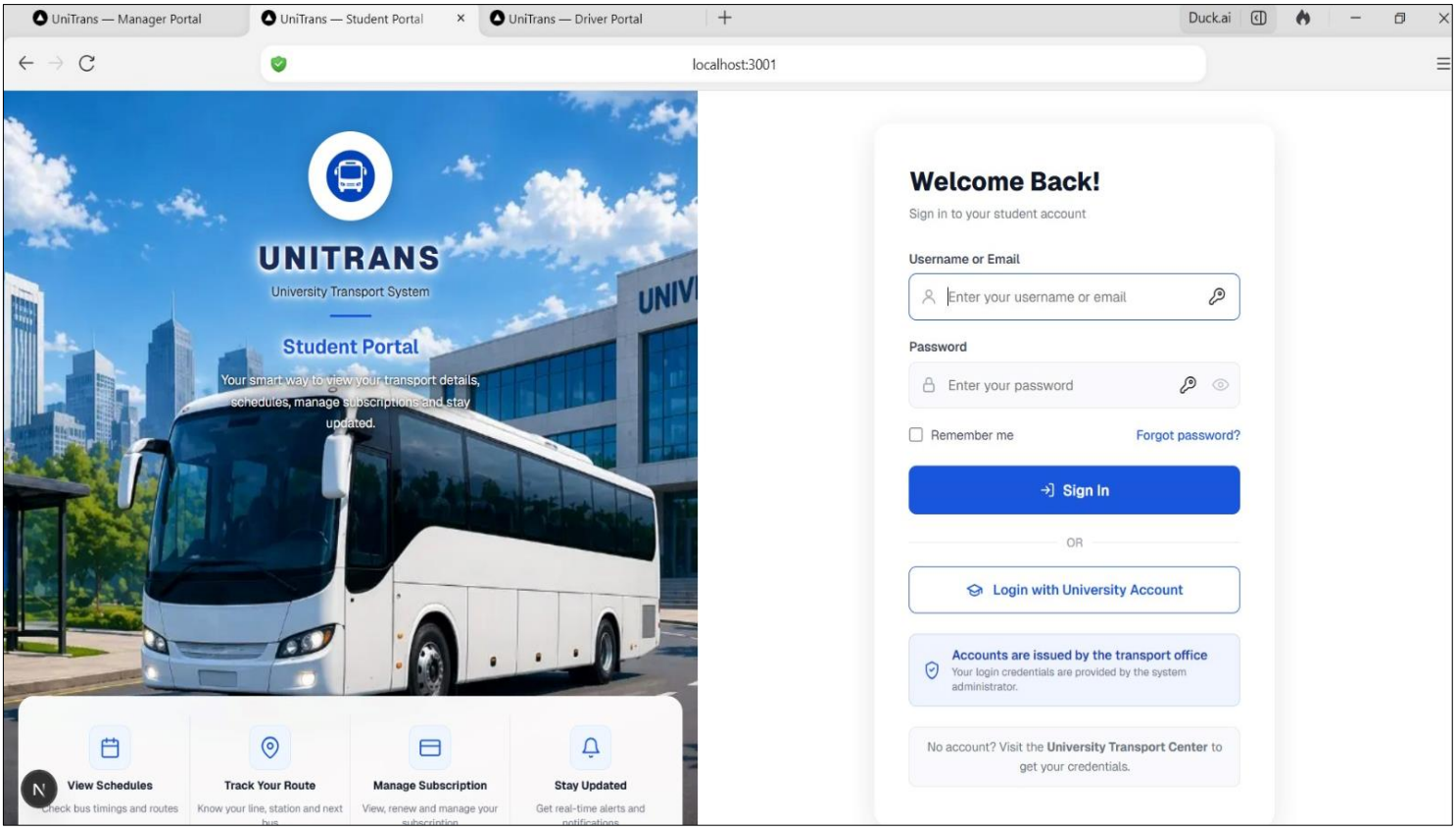
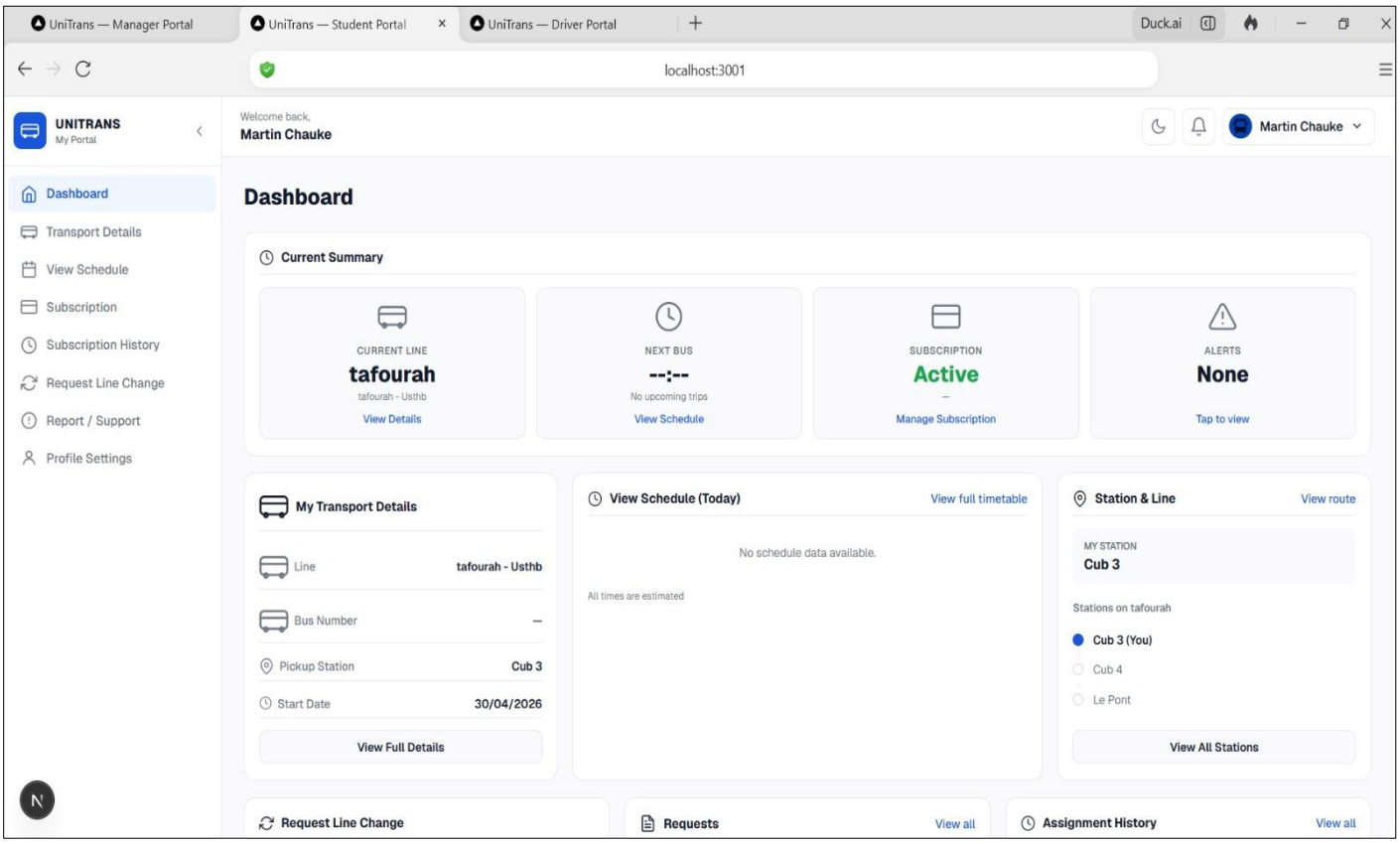


Figure 7: Tableau de bord des étudiants



Pilote

Le tableau de bord du conducteur donne accès à leurs trajets assignés et à leurs horaires quotidiens. Cela leur permet de consulter en temps réel les détails de l'itinéraire et les instructions de transport. Les conducteurs peuvent également signaler les incidents directement via le système pour une intervention administrative rapide.

Figure 8: Page d'atterrissage du conducteur

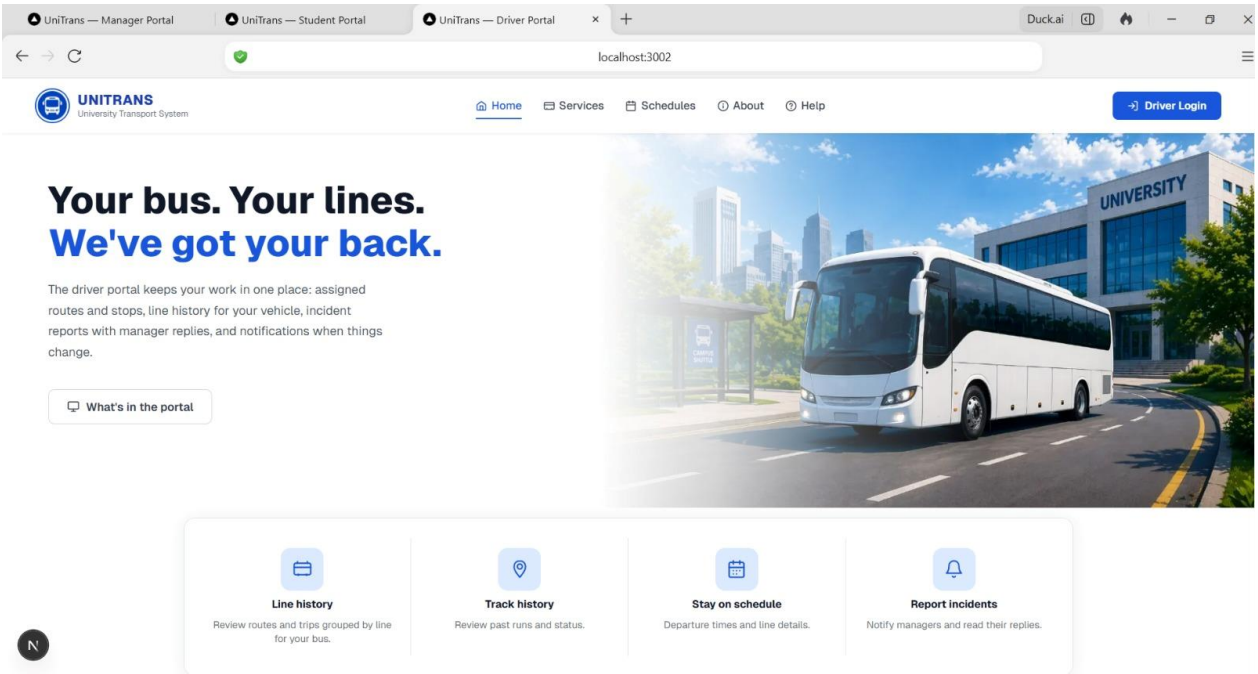


Figure 9: Connexion au pilote

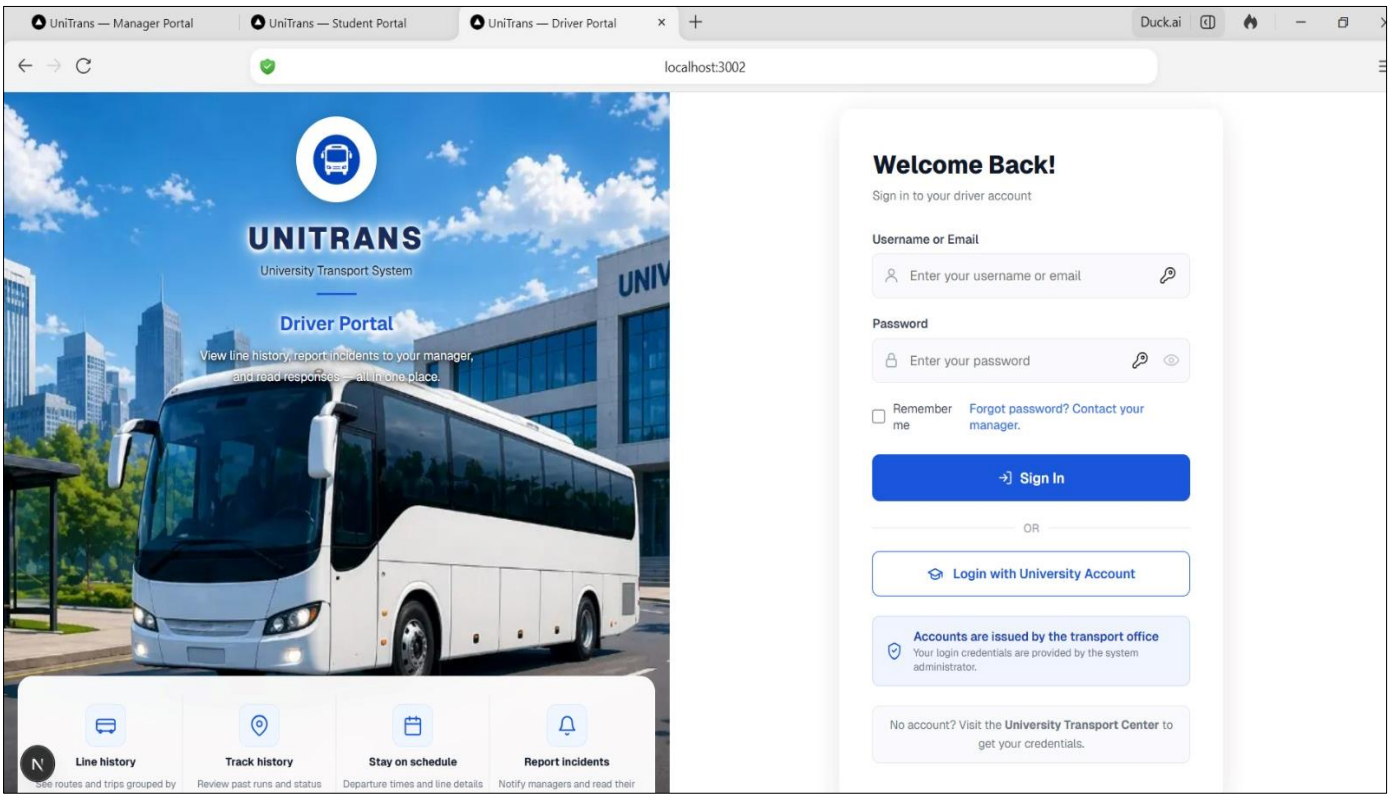
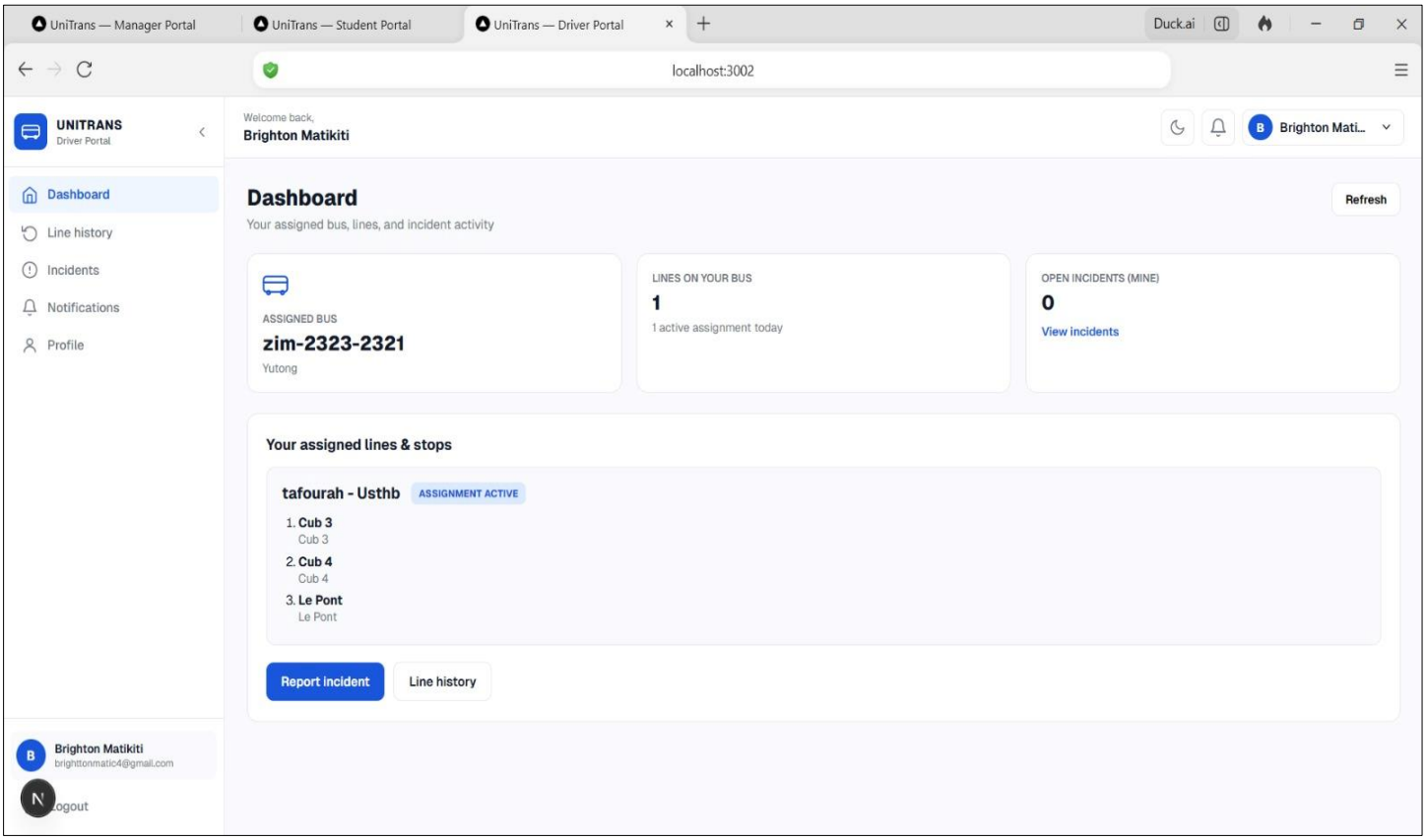


Figure 10: Tableau de bord du conducteur



Entraîneur

Figure 11: Page d'atterrissage du Manager

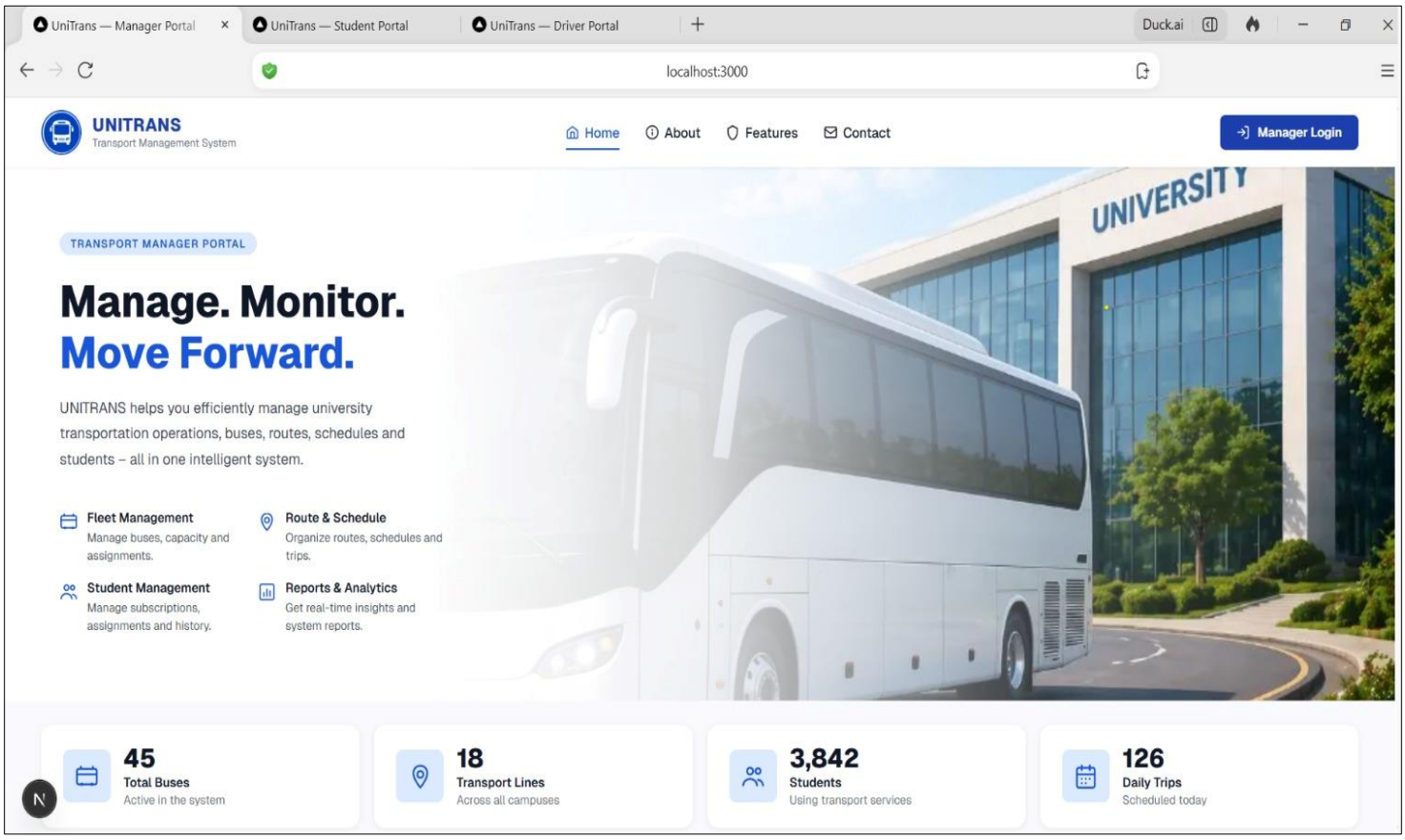


Figure 12: Connexion Manager

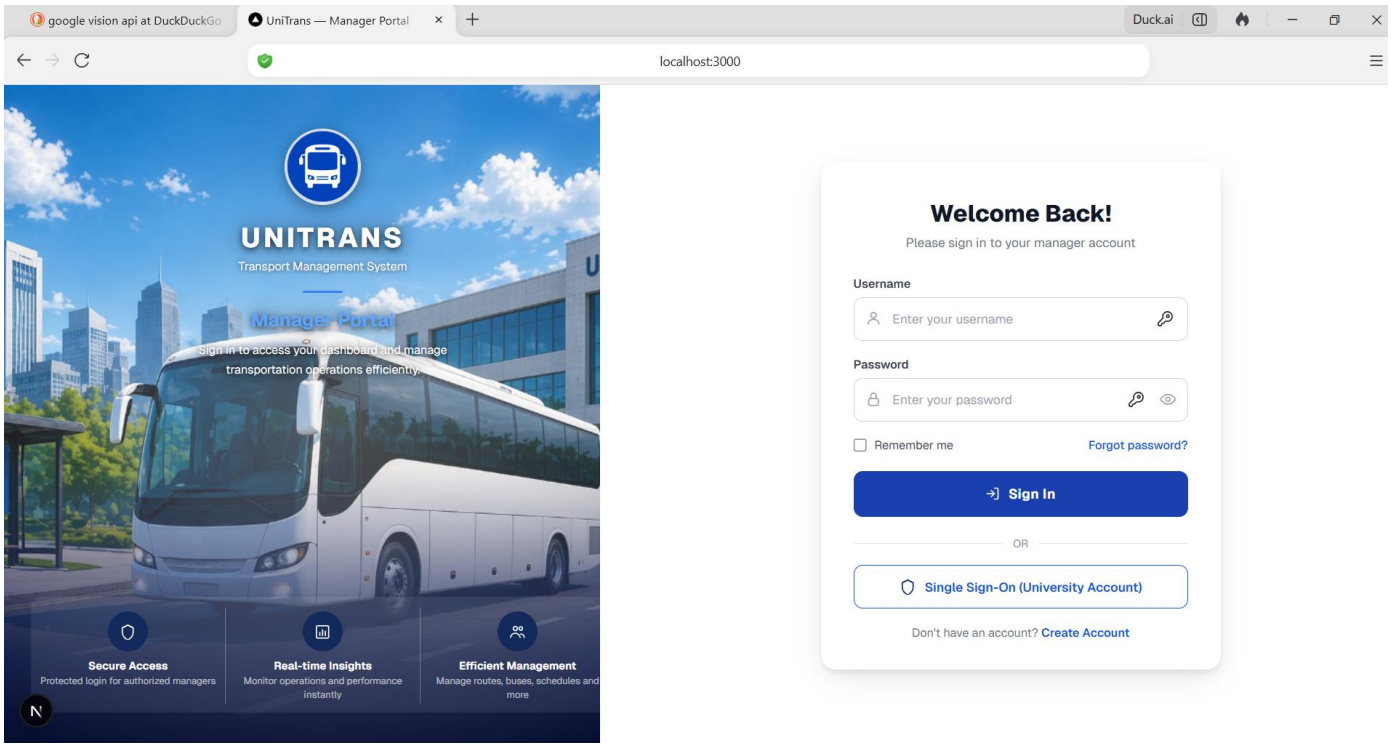
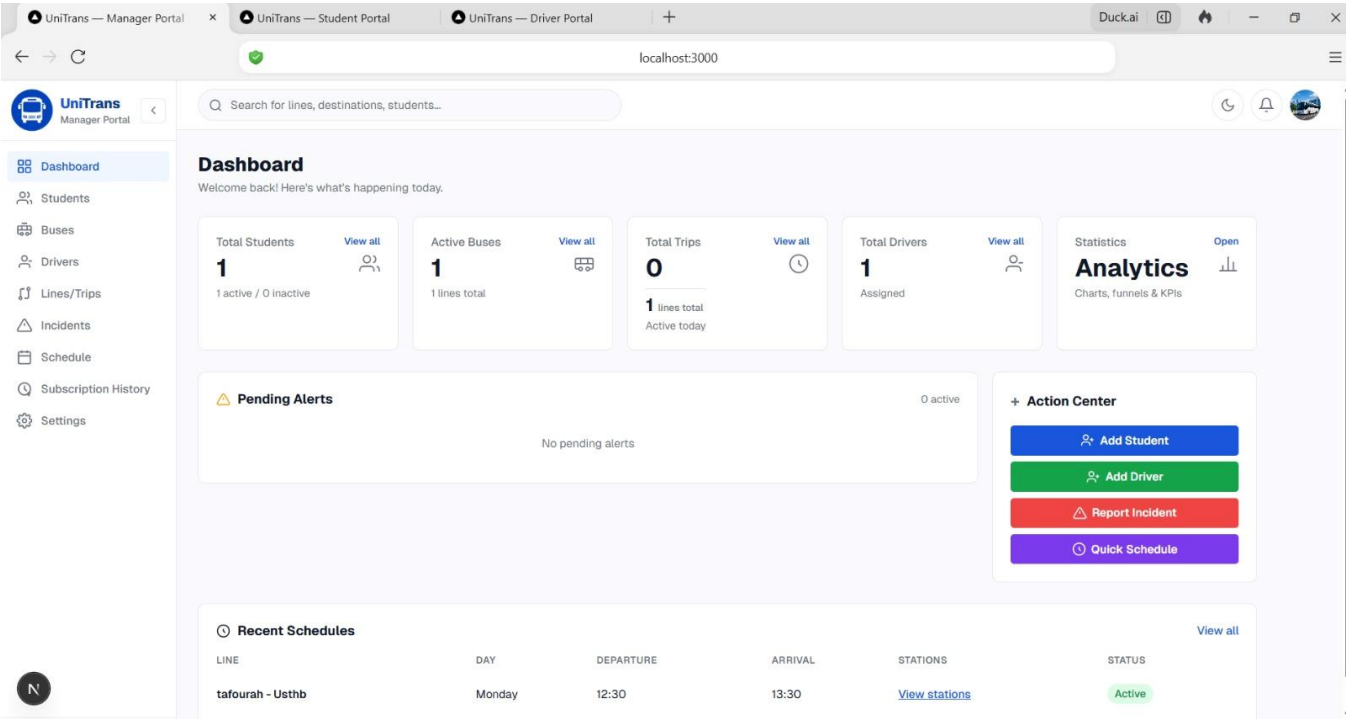


Figure 13: Tableau de bord du manager

Le tableau de bord Manager offre une interface centralisée pour surveiller les opérations de transport, gérer les affectations et accéder efficacement aux statistiques clés du système.



Statistiques

La section statistiques offre un aperçu en temps réel des performances et de l'utilisation du système de transport. Il affiche des indicateurs clés tels que le nombre d'élèves actifs, les bus en service et les déplacements en cours. Cela aide le responsable des transports à surveiller l'efficacité du système et à identifier rapidement les problèmes opérationnels. Il soutient également la prise de décision basée sur les données pour améliorer la qualité des services et l'allocation des ressources.

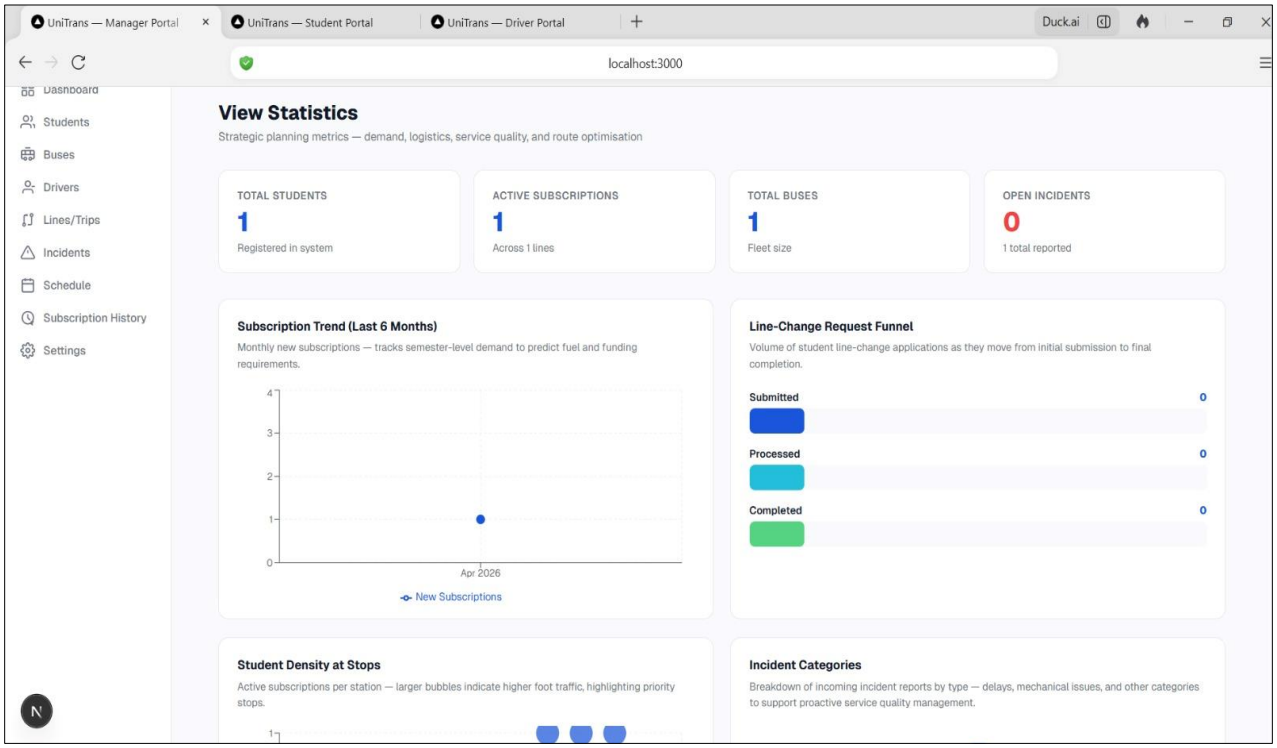
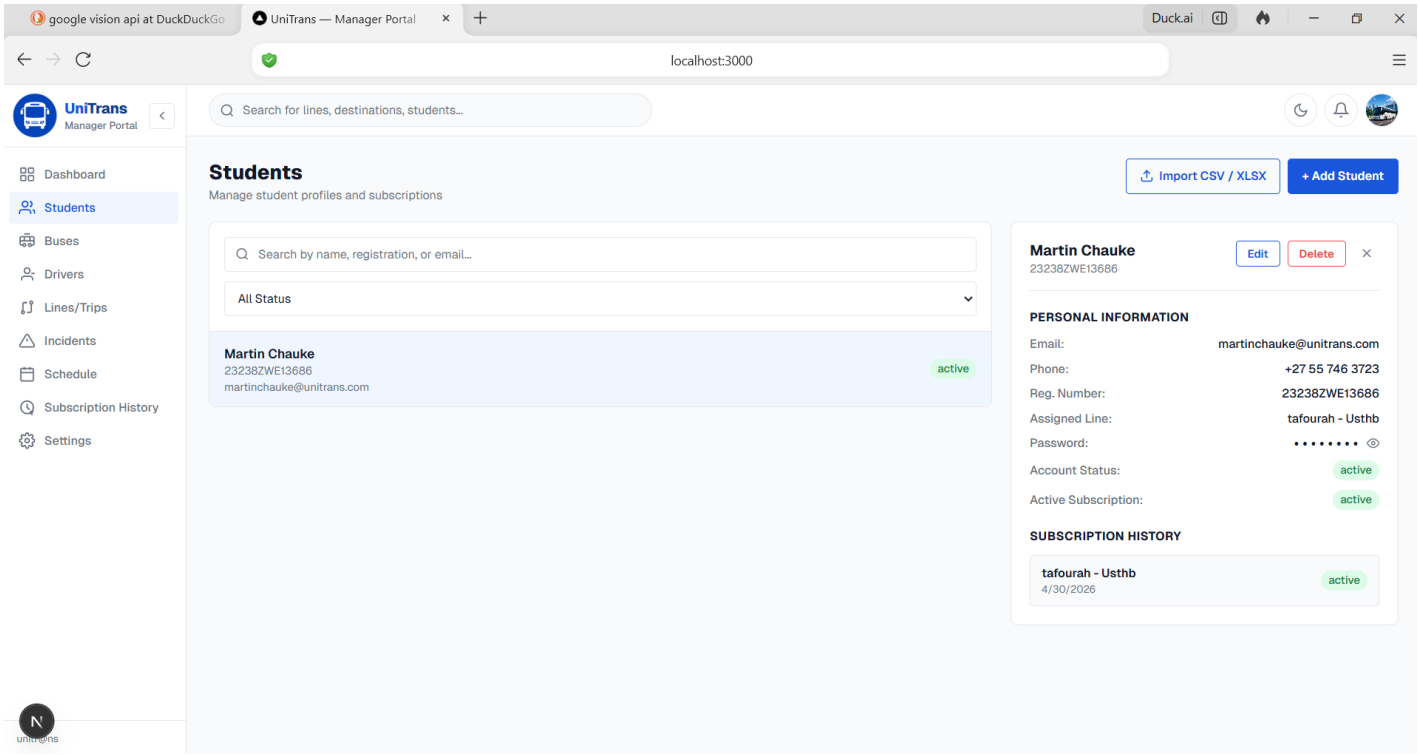


Figure 14: Interface de statistiques système

Figure 15: Voir les élèves par manager.



PostrgreSQL database Schema

Création de tables SQL.

NB : *student_id* est utilisé comme clé primaire car il fournit un identifiant stable généré par le système, tandis que *registration_number* reste une référence administrative unique

USER.

CREATE TABLE « USER » (

user_id CLÉ PRIMAIRE SÉRIE ,

nom VARCHAR(100) NON NULL,

envoyer un email à VARCHAR(150) UNIQUE NOT NULL,

RÔLE VARCHAR(20) NON TEST NUL (

rôle IN ('Admin', 'Gestionnaire des transports', 'Étudiant', 'Conducteur')

);

Étudiant

CREATE TABLE Étudiant (

student_id CLÉ PRIMAIRE SÉRIELLE,

registration_number VARCHAR(50) UNIQUE ET NON NUL,

first_name VARCHAR(100) NON NULL,

last_name VARCHAR(100) NON NULL,

Envoyer un email à VARCHAR(150) UNIQUE NOT NULL,

téléphone VARCHAR (20),

user_id RÉFÉRENCES D'ENTIER *« Utilisateur »* (user_id)

);

Pilote

CREATE TABLE Pilote (

driver_id CLÉ PRIMAIRE SÉRIE ,

first_name VARCHAR(100) NON NULLE,

last_name VARCHAR(100) NON NULLE,

téléphone VARCHAR (20),

license_number VARCHAR(50) UNIQUE ET NON NULLE,

statut VARCHAR(20) PAR défaut *« actif »*,

user_id RÉFÉRENCES D'ENTIERS À « *Utilisateur » (*user_id)

);

Ligne

CREATE TABLE Line (

line_id CLÉ PRIMAIRE DE SÉRIE,

```
 nom VARCHAR(100) NON NULL,
   description TEXTE
);
```

Gare

```
CREATE TABLE Station (
 station_id CLÉ PRIMAIRE DE SÉRIE,
 nom VARCHAR(100) NON NULL,
 adresse VARCHAR (255),
FLOAT de latitude,
   FLOAT de longitude
);
```

Bus

```
CREATE TABLE Bus (
 bus_id CLÉ PRIMAIRE SÉRIE,
   registration_number VARCHAR(50) UNIQUE ET NON NUL,
 modèle VARCHAR (100),
   capacité ENTIER NON NUL,
   statut VARCHAR(20) PAR défaut « actif »
);
```

Row

```
CREATE TABLE Row (
```

```
 row_id CLÉ PRIMAIRE DE SÉRIE,

 row_number ENTIER NON NUL,

 seat_count ENTIER NON NUL,

   bus_id INTEGER NOT NULL RÉFÉRENCE Bus(bus_id) LORS DE LA CASCADE
DELETE

);
```

Schedule

```
CREATE TABLE Schedule (

 schedule_id CLÉ PRIMAIRE DE SÉRIE,

 day_of_week VARCHAR(10) NON NULL,

 departure_time TEMPS NON NÉGLIGE,

   arrival_time TEMPS NON NÉGLIGEABLE,

   direction VARCHAR(20),

   line_id RÉFÉRENCES DE L'ENTIER NON NULL Ligne(line_id)

);
```

Voyage

```
CREATE TABLE Trip (

 trip_id CLÉ PRIMAIRE DE SÉRIE,

   actual_departure HORODATAGE,

   actual_arrival HORODATAGE,

   statut VARCHAR(20) DEFAULT 'programmé',

 schedule_id RÉFÉRENCES ENTIÈRES  Schedule(schedule_id),
```

```
 bus_id RÉFÉRENCES ENTIÈRES  Bus(bus_id),
 driver_id PILOTE DE RÉFÉRENCES ENTIÈRES  (driver_id)
);
```

Incident

```
CREATE TABLE Incident (
 incident_id CLÉ PRIMAIRE DE SÉRIE,
 incident_type VARCHAR (100),
 TEXTE de la DESCRIPTION,
   reported_at HORODATAGE PAR DÉFAUT MAINTENANT(),
 RÉSOLU LE FAUX PAR DÉFAUT BOOLÉEN,
   trip_id RÉFÉRENCES ENTIÈRES NON NULLES Trip(trip_id)
);
```

Subcription

```
CREATE TABLE Subcription (
 subscription_id CLÉ PRIMAIRE DE SÉRIE,
 start_date DATE NON NULLE,
   end_date DATE,
   student_id RÉFÉRENCES ENTIÈRES NON NULLES Étudiant(student_id),
   line_id RÉFÉRENCES DE L'ENTIER NON NULL Ligne(line_id)
);
```

SubscriptionHistory

```
CREATE TABLE SubscriptionHistory (
 subscription_history_id CLÉ PRIMAIRE DE SÉRIE,
   change_date HORODATAGE PAR DÉFAUT MAINTENANT(),
 action_type VARCHAR (30 ans),
TEXTE de la raison,
 student_id RÉFÉRENCES ENTIÈRES  Étudiant(student_id),
   line_id RÉFÉRENCES ENTIÈRES  Ligne(line_id)
);
```

BusAssignment

```
CREATE TABLE BusAssignment (
   bus_assignment_id CLÉ PRIMAIRE SÉRIE ,
 start_date DATE NON NULLE,
   end_date DATE,
   NOTES , TEXTE,
   bus_id RÉFÉRENCES ENTIÈRES NON NULLES Bus(bus_id),
   line_id RÉFÉRENCES ENTIÈRES NON NULLES Ligne(line_id),
   user_id RÉFÉRENCES D'ENTIERS  À « Utilisateur » (user_id)
);
```

Assignation de conducteurs

```
CREATE TABLE DriverAssignment (
 driver_assignment_id CLÉ PRIMAIRE DE SÉRIE,
```

```
 start_date DATE NON NULLE,
   end_date DATE,
   driver_id RÉFÉRENCES ENTIÈRES NON NULLES Driver(driver_id),
   bus_id INTEGER NOT NULL RÉFÉRENCES Bus(bus_id)
);
```

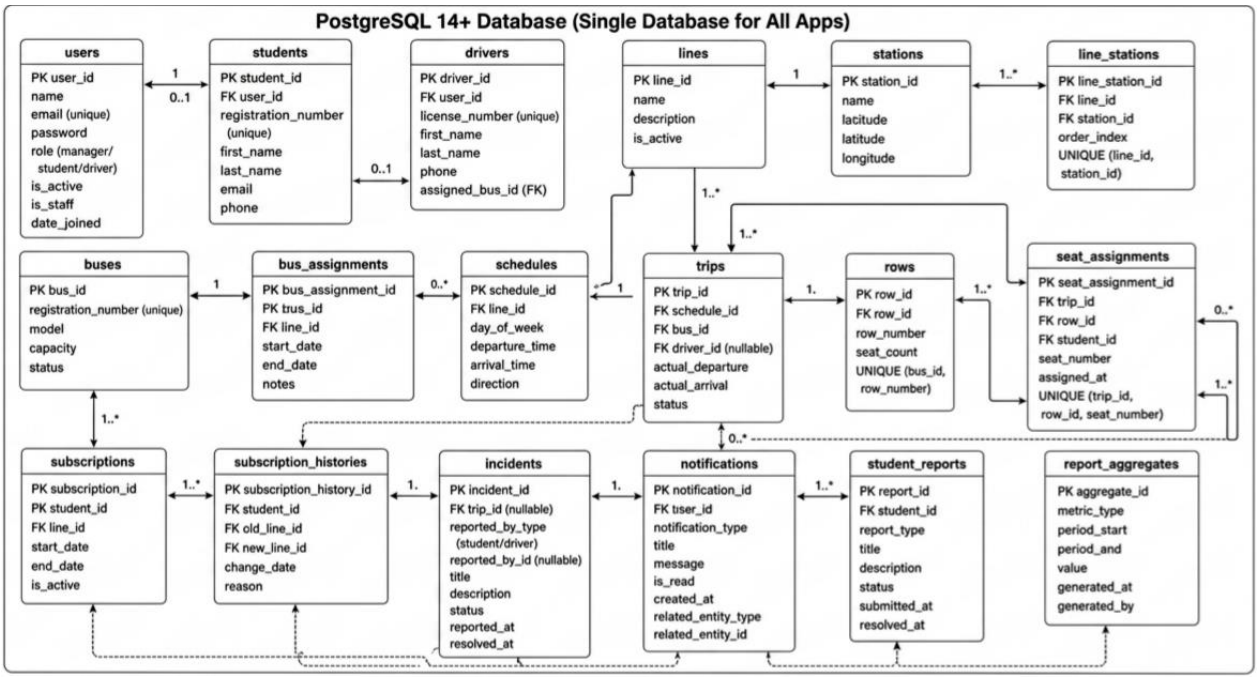
Attribution des sièges

```
CRÉER TABLE SeatAssignment (
   seat_assignment_id CLÉ PRIMAIRE SÉRIE ,
   assigned_at HORODATAGE PAR DÉFAUT MAINTENANT(),
   student_id RÉFÉRENCES ENTIÈRES NON NULLES Étudiant(student_id),
   trip_id RÉFÉRENCES ENTIÈRES NON NULLES Trip(trip_id),
   row_id RÉFÉRENCES DE L'ENTIER NON NULL Ligne(row_id)
);
```

SERVICES

```
CREAT TABLE  SERVICES (
   line_id RÉFÉRENCES ENTIÈRES NON NULLES Ligne(line_id),
   station_id RÉFÉRENCES DE NOMBRE ENTIER ET NON NUL Station(station_id),
 order_index ENTIER NON NUL,
   CLÉ PRIMAIRE (line_id, station_id)
);
```

Figure 16: Schéma de base de données



Conclusion

Le système *UniTrans* a été conçu pour offrir une solution efficace et structurée pour la gestion des services de transport universitaire. L'objectif principal de ce projet était de remplacer la gestion manuelle et non organisée des transports par un système de base de données centralisé.

Au fil des phases d'analyse et de conception, le système a été modélisé à l'aide de conception conceptuelle (MCD), de modèle relationnel (MLD), et normalisé jusqu'au BCNF, assurant la cohérence des données et réduisant la redondance.

La mise en place du système permet une meilleure gestion des élèves, des lignes de transport, des bus, des chauffeurs, des abonnements et des incidents.

Dépôt GitHub : *UniTrans_Source_Code*

Références.

[1] Silberschatz, A., Korth, H. F., Sudarshan, S.

[2] Elmasri, R., Navathe, S. B.

[3] Date, C. J.

[4] Connolly, T., Begg, C.

[5] Documentation PostgreSQL

[6] Manuel de référence du langage Oracle SQL.